Zonoth

SESSION 02 · 2H

Building images

From a Dockerfile nobody should ship to Pinboard's real one: 13 MB, non-root, reproducible, signed for what it contains.

Dockerfile | Layers & cache | Multi-stage & base images | Publish & supply chain

Lab 02

SESSION 02

What we'll cover today

1 · The Dockerfile

The build context, `.dockerignore`, and every instruction you need — `FROM` through `HEALTHCHECK` — on a first, honest, terrible Dockerfile.

2 · Layers & the cache

How BuildKit turns instructions into layers, what invalidates the cache, and why instruction order decides whether a rebuild takes 2 s or 2 min.

3 · Small, safe, published

Multi-stage builds, the base-image ladder down to distroless, non-root users, then tagging, multi-platform, SBOM and `docker scout`.

4 · Lab 02

Write Pinboard's multi-stage Dockerfile yourself, build `1.0` and `1.1`, build the web image, and compare the sizes.

01

The Dockerfile

A recipe read top to bottom by a builder — where every line you write becomes something a future colleague has to download.

THE FLOW

What `docker build` actually does

01
Context
The folder is sent to the builder

→

02
BuildKit
Parses the Dockerfile into a graph of steps

→

03
Layers
Runs only the steps not already cached

→

04
Image
Manifest + config + layers, tagged locally

- **BuildKit** is the default builder in Docker Engine 28 — parallel steps, better caching, cache mounts, secrets
- `docker buildx build` is the same engine with the full flag set; `docker build` is the friendly front door
- The output is a local image; nothing leaves your machine until you `push`

INPUTS

The build context and `.dockerignore`

- The last argument to `docker build` is a **folder**, and all of it is uploaded to the builder
- `COPY` can only read from inside that folder — never from `../`
- `.dockerignore` keeps junk out: faster builds, smaller images, no leaked secrets
- It also protects the cache — a changed `.git` should not rebuild your app

BASH

```
$ docker build -t pinboard-api:1.0 ./pinboard-api
#1 [internal] load build context
#1 transferring context: 48.2kB done
```

TEXT

```
# .dockerignore — a typical example
.git
.gitignore
*.md
.env
Dockerfile
dist/
**/testdata/
```

ANATOMY

The instructions you actually need

INSTRUCTION	WHAT IT DOES	IN PINBOARD
`FROM`	Starts a stage from a base image	`golang:1.25-alpine AS build`
`WORKDIR`	Sets (and creates) the current directory	`/src`
`COPY`	Copies from context or another stage	`go.mod go.sum ./`
`RUN`	Executes a command, commits a layer	`go mod download`
`ARG` / `ENV`	Build-time variable / runtime variable	`ARG APP_VERSION`, `ENV PORT`
`USER`	Which UID the process runs as	`65532:65532`
`ENTRYPOINT` / `CMD`	The process / its default arguments	`["/pinboard-api"]`
`HEALTHCHECK`	How the engine tests liveness	`wget -qO- …/healthz` (web image)

💡 `LABEL` adds OCI metadata, `EXPOSE` documents a port. Prefer `COPY` over `ADD`: `ADD` also fetches URLs and auto-extracts archives, which surprises people.

START HERE

A first Dockerfile — and everything wrong with it

DOCKERFILE

```
FROM golang:1.25-alpine
WORKDIR /src
COPY . .
ENV CGO_ENABLED=0
RUN go build -o /pinboard-api .
EXPOSE 8080
CMD ["/pinboard-api"]
```

⚠ **It works.** It also produces a **412 MB** image containing the Go compiler, your source code and a package manager — running as **root**.

- `COPY . .` before the build means **any** file change re-downloads every dependency
- No version metadata, no `USER`, no `.dockerignore`, nothing pinned

02

Layers and the cache

Instruction order is not style. It is the difference between a two-second rebuild and a two-minute one, fifty times a day.

MECHANICS

Every instruction is a cached step

- Instructions that change the filesystem produce a **layer**; the rest only change config
- Layers are content-addressed, so identical layers are stored and pulled once
- BuildKit reuses a step when its **cache key** matches — and prints `CACHED`
- `docker history` tells you which line cost you which megabytes

```
BASH
 $ docker history pinboard-api:1.0
IMAGE      CREATED BY                      SIZE
<missing>  ENTRYPOINT ["/pinboard-api"]     0B
<missing>  USER 65532:65532                 0B
<missing>  COPY /out/pinboard-api / # bui  9.6MB
<missing>  /bin/sh -c #(nop) LABEL org.op   0B
<missing>  distroless base                 1.9MB
```

THE RULE THAT MATTERS

What invalidates the cache

- Steps are checked **top to bottom**; the first miss rebuilds everything after it
- `COPY` / `ADD`: the key is a **checksum of the copied files** — content, mode, path
- `RUN`: the key is the **command string**, not its result — so `apt-get update` alone can serve week-old package lists
- Changing an `ARG` or `ENV` a stage uses invalidates from that point
- A new base image digest invalidates the whole stage

⚠ Always combine `apt-get update` and `apt-get install` in **one** `RUN`, or you will install yesterday's packages forever.

💡 Put the **rarely changing** things first and the **constantly changing** things — your source code — last.

IN PRACTICE

Ordering Pinboard's build stage

DOCKERFILE

```
# 1. dependencies change rarely — cache them on their own layer
COPY go.mod go.sum ./
RUN --mount=type=cache,target=/go/pkg/mod go mod download

# 2. the source changes constantly — copy it last
COPY . .
RUN --mount=type=cache,target=/go/pkg/mod \
    --mount=type=cache,target=/root/.cache/go-build \
    CGO_ENABLED=0 go build -o /out/pinboard-api .
```

- Editing a `.go` file now rebuilds only the last step — dependencies stay `CACHED`
- `--mount=type=cache` keeps the module and compiler caches **outside** the image, across builds

LIVE DEMO 1 · 10 MIN

Watch the cache work — and break

Build the naive Dockerfile, read `docker history`, change one line of Go, rebuild, and watch every dependency download again. Then make the same change against the ordered Dockerfile and watch the dependency layer stay cached.

What you'll see · 412 MB · CACHED lines · a busted cache

Files · demos/session02/naive.Dockerfile · src/pinboard-api/

Duration · 10 min

03

Small and safe images

412 MB to 13 MB with one idea: build in one image, ship from another.

THE BIG IDEA

Multi-stage builds

- Several `FROM`s in one file; each starts a fresh stage
- `COPY --from=build` takes **only the artefact** across
- The toolchain, the source and the module cache never reach the final image

DOCKERFILE

```
# syntax=docker/dockerfile:1
FROM golang:1.25-alpine AS build
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
ARG APP_VERSION=dev
RUN CGO_ENABLED=0 go build -trimpath -o /out/pinboard-api \
  -ldflags="-X main.defaultVersion=${APP_VERSION}" .
FROM gcr.io/distroless/static-debian12:nonroot AS runtime
COPY --from=build /out/pinboard-api /pinboard-api
EXPOSE 8080
USER 65532:65532
ENTRYPOINT ["/pinboard-api"]
```

STAGE CONTROL

Building part of the file: `--target`

```bash
# stop at the build stage — a shell, the compiler, the source
docker build --target build -t pinboard-api:builder ./pinboard-api
docker run --rm -it pinboard-api:builder sh

# the real thing
docker build -t pinboard-api:1.0 --build-arg APP_VERSION=1.0 ./pinboard-api
```

- Handy for a `test` or `lint` stage that CI builds and production never ships
- BuildKit builds independent stages **in parallel** and skips stages the target does not need
- A debug image built from the same stage is far better than adding a shell to production

CHOOSING A BASE

The base-image ladder

BASE	SIZE	CONTAINS	CHOOSE WHEN
`golang:1.25` / `debian:12`	~350–800 MB	Full distro + toolchain	Build stages only
`debian:12-slim`	~75 MB	apt, shell, glibc	You need apt packages at runtime
`alpine:3.22`	~8 MB	busybox, apk, musl	Small **and** you want a shell
`distroless/static`	~2 MB	CA certs, tzdata, nonroot user	Static binaries — **Pinboard's choice**
`scratch`	0 B	Nothing at all	You will add certs and a user yourself

⚠ Alpine uses **musl**, not glibc — a real source of surprises with DNS resolution and pre-compiled binaries. Test, don't assume.

LEAST PRIVILEGE

Don't run as root

- Root in a container is **root on the host kernel** (unless user namespaces are on)
- Distroless `:nonroot` = UID **65532**; elsewhere set `USER`
- Kubernetes refuses root images with `runAsNonRoot` (S09)
- Non-root can't bind ports < 1024 — hence **8080**

BASH

```
$ docker image inspect -f '{{.Config.User}}' pinboard-api:1.0
65532:65532

$ docker run --rm nginx:1.28-alpine id
uid=0(root) gid=0(root)
```

Set `USER` as the **last** instruction, after anything that needs to write during the build.

THE PAYOFF

Same binary, one twentieth the size

412 MB
Naive

`golang:1.25-alpine` with the source, the compiler and the module cache still inside. Runs as root.

23 MB
Alpine runtime

Multi-stage onto `alpine:3.22`. Has a shell — convenient, and an attack surface.

13 MB
Distroless

Binary + certs + nonroot user. Nothing to exec into, nothing to exploit.

Multiply by every node that pulls it, every rollout, every CI run. Small images make session 06's rolling updates feel instant.

LIVE DEMO 2 · 9 MIN

Naive to multi-stage, side by side

Build the real multi-stage Dockerfile, compare the two images in `docker image ls`, prove there is no shell in the distroless one, then run it and call `/api/info`.

What you'll see · 412 MB vs 13 MB · `--target` · no shell

Files · demos/session02/naive.Dockerfile · src/pinboard-api/Dockerfile

Duration · 9 min

04

Publish and prove

Getting the image to a registry — and being able to say, later, exactly what is in it and where it came from.

`docker init` — don't start from a blank file

```bash
$ cd my-go-service && docker init

? What application platform does your project use?  Go
? What version of Go do you want to use?           1.25
? What port does your server listen on?            8080

Created: .dockerignore, Dockerfile, compose.yaml, README.Docker.md
```

- Ships with Docker Desktop. Generates a sane multi-stage Dockerfile, a `.dockerignore` and a Compose file for Go, Node, Python, Rust, PHP and ASP.NET
- Treat it as a **draft**: check the base image, the `USER`, the pinning and the health check before committing

SHIP IT

Tag and push

```bash
 docker login ghcr.io                        # credentials go to ~/.docker/config.json
docker tag pinboard-api:1.0 ghcr.io/zonoth/pinboard-api:1.0
docker push ghcr.io/zonoth/pinboard-api:1.0

# or build straight to the registry
docker buildx build --push -t ghcr.io/zonoth/pinboard-api:1.0 ./pinboard-api
```

- A tag is just a pointer — the same image can carry `1.0`, `1` and a commit SHA at once
- `push` uploads only the layers the registry is missing
- OCI labels (`org.opencontainers.image.version`, `.source`, `.revision`) travel **inside** the image — Pinboard sets them in its `LABEL` block

Multi-platform images

```bash
 docker buildx create --name multi --driver docker-container
docker buildx build --builder multi --platform linux/amd64,linux/arm64 \
  -t ghcr.io/zonoth/pinboard-api:1.0 --push ./pinboard-api
docker buildx imagetools inspect ghcr.io/zonoth/pinboard-api:1.0
```

- One tag becomes an **image index** listing a manifest per architecture; the daemon picks the right one automatically
- Go cross-compiles for free — `FROM --platform=$BUILDPLATFORM` plus `GOARCH=$TARGETARCH` in the build stage instead of QEMU emulation, which is slow
- Multi-platform results cannot sit in the classic local image store: use `--push`, or `--load` a single platform

SUPPLY CHAIN

SBOM, provenance, and a scan

```bash
docker buildx build --builder multi --sbom=true --provenance=mode=max \
-t ghcr.io/zonoth/pinboard-api:1.0 --push ./pinboard-api

docker scout quickview pinboard-api:1.0
docker scout cves pinboard-api:1.0 --only-severity critical,high
```

- **SBOM** — the bill of materials: every package inside the image, in SPDX form
- **Provenance** — how it was built: source, builder, arguments (SLSA-style attestation)
- Both attach to the image index as attestations and travel with the image
- `docker scout` compares those contents against advisory data and suggests a better base

BEFORE YOU COMMIT

Dockerfile best-practice checklist

Size & base

Multi-stage build · smallest base that works · pin the tag (ideally the digest) · no compilers or package managers in the runtime stage.

Cache & context

Dependencies before source · one `RUN` for update+install · a real `.dockerignore` · cache mounts instead of copied caches.

Security

`USER` non-root · no secrets in layers (use `--mount=type=secret`) · only the ports you need · a scan in CI.

Metadata & runtime

OCI `LABEL`s · version via `ARG` · `ENTRYPOINT` in exec form · `HEALTHCHECK` · config from env, never baked in.

LIVE DEMO 4 · 7 MIN

Attest and scan the image we just built

Rebuild `pinboard-api:1.0` for two architectures with an SBOM and provenance, inspect the resulting index — and, if time allows, run `docker scout quickview` on it and on the naive image for contrast.

What you'll see · image index · attestations · a CVE table **Files** · src/pinboard-api/ **Duration** · 7 min

LAB 02 · 30–40 MIN

Build the Pinboard images

Fill in the blanks of the multi-stage Dockerfile for `pinboard-api`, build `1.0` and `1.1` from the same source with a build argument, build `pinboard-web`, run the API and call `/api/info` to see the version you injected.

You'll practise: writing a Dockerfile · `.dockerignore` · `--build-arg` · `image ls` & `history` · `run --rm -p` · optional `tag` + `push`

Key takeaways

✓ Order for the cache

Rarely-changing steps first, source last. The first cache miss rebuilds everything below it.

✓ Build here, ship from there

Multi-stage plus a minimal base turned 412 MB into 13 MB with the same binary.

✓ Non-root, pinned, described

`USER`, a pinned base, OCI labels, an SBOM and a scan — the 2026 minimum.

✓ Next up

Session 03 — running containers well: configuration, signals and PID 1, volumes and networking.

Zonoth

Questions?

Up next — Session 03: Running containers well. Configuration, storage and networking, with Pinboard wired together by hand.